

**SpyGlass[®] DFT DSM Optimized
Design
Submethodology (for GuideWare
V-2023.12-SP1)**

Version V-2023.12-SP1, March 2024



Copyright Notice and Proprietary Information

©2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <http://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Report an Error

The SpyGlass Technical Publications team welcomes your feedback and suggestions on this publication. Please provide specific feedback and, if possible, attach a snapshot. Send your feedback to spyglass_support@synopsys.com.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Contents

Preface.....	7
About This Book	7
Contents of This Book	8
Typographical Conventions	9
 Using DFT-DSM Sub-Methodology	 11
The Need for DFT-DSM Optimized Design	11
Introduction.....	12
Objective.....	12
Tool Versions.....	12
References	12
Prerequisites.....	12
DFT Overview.....	13
DFT for Stuck-at Testing	13
Scan Design.....	13
Stuck-at Test Application	13
Test Clock Considerations	14
DFT for Transition Testing	14
Transition Test Design	15
ATE Generated Clocks	16
On-Chip Clock Generation.....	16
GuideWare Methodology for DFT-DSM.....	17
Approach.....	19
The scan_ready goal	19
DFT-DSM Clocks	19
Best Practice	23
Step-by-Step Solution	31
Goal: dft_scan_ready.....	31
Goal: dft_dsm_best_practice.....	37
Using the Coverage_Audit report to ensure test operation	46

Preface

About This Book

The SpyGlass DFT DSM Optimized Design Sub-Methodology guide describes using the SpyGlass DFT DSM methodology.

Contents of This Book

The SpyGlass DFT DSM Optimized Design Sub-Methodology guide consists of the following sections:

Section	Description
Using DFT-DSM Sub-Methodology	About using SpyGlass DFT DSM sub-methodology

Typographical Conventions

This document uses the following typographical conventions:

To indicate	Convention Used
Program code	OUT <= IN;
Object names	OUT
Variables representing objects names	<sig-name>
Message	Active low signal name '<sig-name>' must end with _X.
Message location	OUT <= IN;
Reworked example with message removed	OUT_X <= IN;
Important Information	NOTE: This rule...

The following table describes the syntax used in this document:

Syntax	Description
[] (Square brackets)	An optional entry
{ } (Curly braces)	An entry that can be specified once or multiple times
(Vertical bar)	A list of choices out of which you can choose one
. . . (Horizontal ellipsis)	Other options that you can specify

Using DFT-DSM Sub-Methodology

The Need for DFT-DSM Optimized Design

Designing for test for deep submicron (DFT-DSM) designs is quite prevalent today. Manufacturing test is dominated by patterns automatically generated by ATPG (automatic test pattern generation) tools. To operate effectively, these tools require that the circuits be correctly designed for testing. This document and the corresponding SpyGlass DFT software directly address this issue.

Introduction

Objective

This document introduces a method to run SpyGlass DFT DSM.

Tool Versions

- SPYGLASS: Version V-2023.12-SP1
- DFT_DSM: Version V-2023.12-SP1
- GuideWare: Version V-2023.12

References

- SpyGlass DFT User Guide
- SpyGlass DFT DSM User Guide
- Design Read Methodology Document
- SpyGlass DFT Methodology Document
- SpyGlass SoC Methodology User Guide

Prerequisites

You are expected to have basic knowledge of SpyGlass operations and to have a thorough understanding of scan design and SpyGlass DFT.

SpyGlass DFT DSM requires a clean design read and high stuck-at coverage by running the DFT policy. Please refer to the SpyGlass Design Read Methodology and SpyGlass DFT Methodology documents for details.

DFT Overview

This section briefly describes and contrasts DFT for stuck-at testing and DFT for transition testing.

DFT for Stuck-at Testing

The ATPG tools in common use today are much more efficient for processing combinational circuits than for sequential circuits. As a result, the primary DFT approach commonly used for testing stuck-at faults is scan design. Key objectives of this design method are listed in the subsequent sections.

Scan Design

Design for scan involves ensuring that test clocks control as many registers as possible and that any sets and resets to these registers can be disabled during shift mode.

If these conditions are achieved then sequential circuits can be treated as if they were combinational.

Stuck-at Test Application

If we assume that the flip-flops in [Figure 1](#) are scanned, then the AND gate can be tested by setting a state in U1 and U2, via scan and then firing one clock pulse to capture the results in U3. Only one “capture” clock pulse is needed for each scan test. Since the stuck-at failure mode is assumed to be static, timing of the capture pulse is not critical so long as the circuit reaches a stable condition before the capture clock pulse.

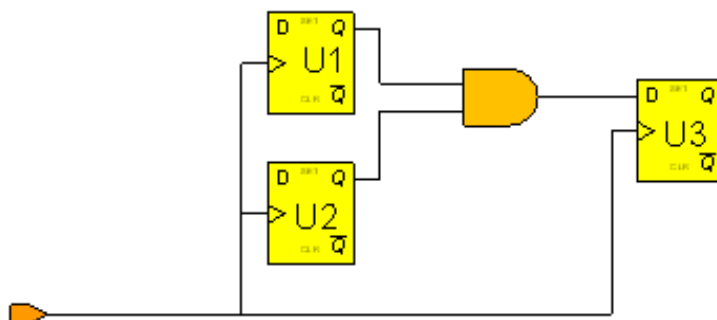


FIGURE 1. Simple stuck-at test example

Test Clock Considerations

The waveforms in [Figure 2](#) are typically used to shift tests into a circuit and to capture test data using these chains. The capture clock must be operated in system mode and therefore the circuit must be designed to guarantee that capture pulses reach the scan flip-flops regardless of scan-in state.

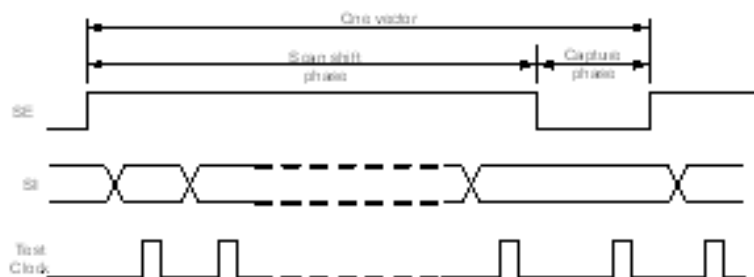


FIGURE 2. Waveforms for stuck-at testing

DFT for Transition Testing

DFT for transition testing starts with scan design, so all the requirements

for scan also apply here. However, in contrast to DFT for stuck-at testing, DFT for transition testing has additional requirements in test clock handling.

Transition Test Design

If we assume that all flip-flops in [Figure 3](#) are scannable, then transition tests for the combinational logic begin with scanning the flip-flops (U1 and U2) that directly feed the combinational logic. The flip-flops that control the next state of U1 and U2 (U4 and U5) are also scanned in.

In order to perform transition tests on the AND gate, a $0 \Rightarrow 1$ transition or a $1 \Rightarrow 0$ transition must pass through the AND gate. In this simple case, a $0 \Rightarrow 1$ on the AND can be created by scanning a 0 into U1 and a 1 into U4. Two test clock pulses are required. The first pulse causes U1 to change state from 0 to 1. If the test is properly setup and if the AND is functioning correctly, this transition will appear on the D input to U3. A second clock pulse captures the test result into U3. Typical transition test waveforms are illustrated in [Figure 4](#).

A critical consideration is that the two capture test clock pulses must occur at system speed.

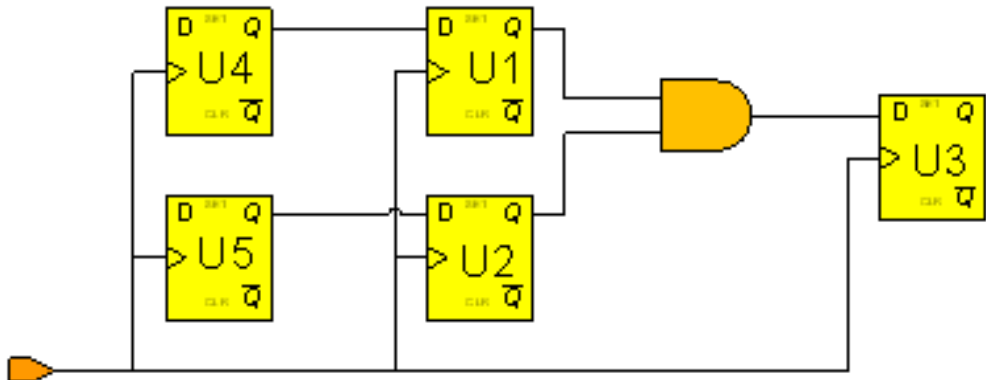


FIGURE 3. Simple Transition Test Example

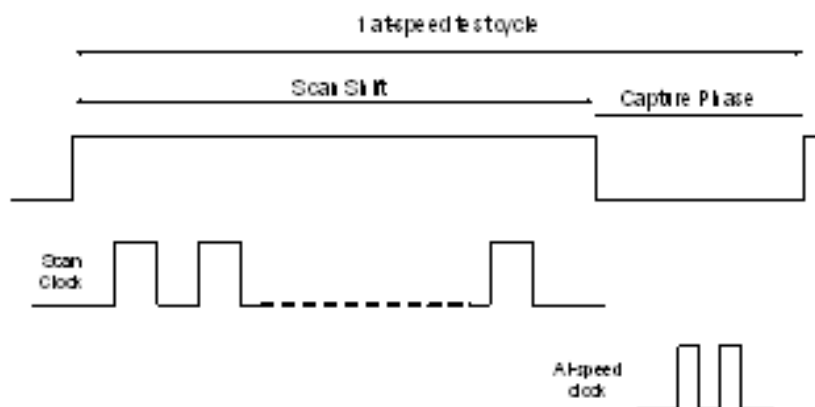


FIGURE 4. Waveforms for Transition Testing

Two methods are typically used to generate at-speed test clocks.

ATE Generated Clocks

In this method, the at-speed test clocks are generated by the test machine. Therefore, this method is similar to the current DFT (stuck-at test) behavior where PLLs are by-passed so that scan shifting and test capture can be performed by the test machine clocks.

The problem with this method is that only expensive test machines are capable of generating the precision, high speed clocks necessary for at-speed testing. Moreover, if a design has a large number of asynchronous system clocks, it may not be practical to generate the at-speed test clocks from the test equipment.

On-Chip Clock Generation

In this method, on-chip PLLs are designed with the necessary clock rates and pulse controls so that the two capture clock pulses are produced when performing transition tests.

GuideWare Methodology for DFT-DSM

The SpyGlass GuideWare methodology describes two fields of use: Block and SoC design.

- **Block field of use:** In this field of use, checks and goals are organized to align with the evolution and the maturity of new or re-used RTL blocks.
- **SoC field of use:** The SoC integration phase includes stitching of the new RTL blocks or IPs. This field of use is divided into the following stages: Initial RTL, RTL handoff, netlist handoff and layout Handoff.

For a typical RTL to layout design flow, SpyGlass DFT DSM offers the following goals:

- `dft_dsm_best_practice` - Ensures that best practice design guidelines are followed which in turn ensures that the coverage numbers will be higher. It provides an early estimation of the transition coverage for the design and gives a detailed reports on how it can be improved by addressing different issues in the design

The following tables show how the GuideWare fields of use correspond to the DFT-DSM goals. The tables shown below use the following conventions:

- M: Mandatory
- O: Optional (Optional goals can be accessed from Tools/Methodology_Configuration menu in Console GUI)

DFT: Block		
GuideWare Stage	Goals	dft_dsm_best_practice
rtl_handoff		M
netlist_handoff		M

DFT: SoC		
GuideWare Stage	Goals	dft_dsm_best_practice
rtl_handoff		M
netlist_handoff		M
layout_handoff		M

Approach

The approach will follow the steps illustrated in [Figure 5](#). Processing a design in the order shown helps to reduce the time to complete the job and eliminate unnecessary interactions with the tool. Each block will be covered in the following subsections.

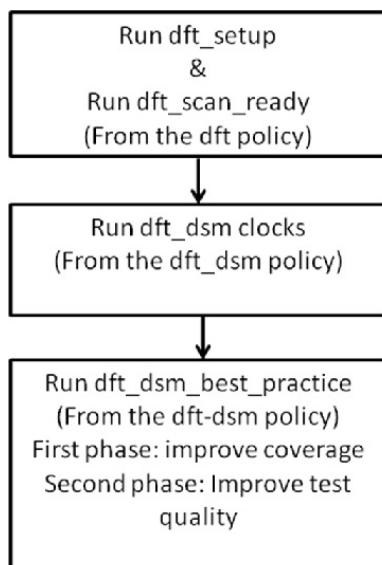


FIGURE 5. Approach

The scan_ready goal

Run the scan_ready goal to identify test clocks, test mode signals and other test related constraints.

Use the process described in the SpyGlass DFT Sub-methodology Guide to achieve high stuck-at test coverage. Use the associated SGDC file as the starting point for achieving high transition test coverage.

DFT-DSM Clocks

The objective of this step is to check the connectivity of clocks that will be used for at-speed testing and any associated PLLs and captureATspeed testmode requirements.

PLLs

All PLLs in a design should be declared in `p11` constraints. For the example shown in [Figure 6](#), `myPLL` should be declared as a PLL so that the simulation engine can propagate clock pulses through the PLL.

In this case, the following constraints should be used:

```
clock -name rootclk -testclock -atspeed
pll -name myPLL -clkkin ref_clk -clkout clkout
```

If a design does not contain any PLLs, it is sufficient to just declare the at-speed test clocks.

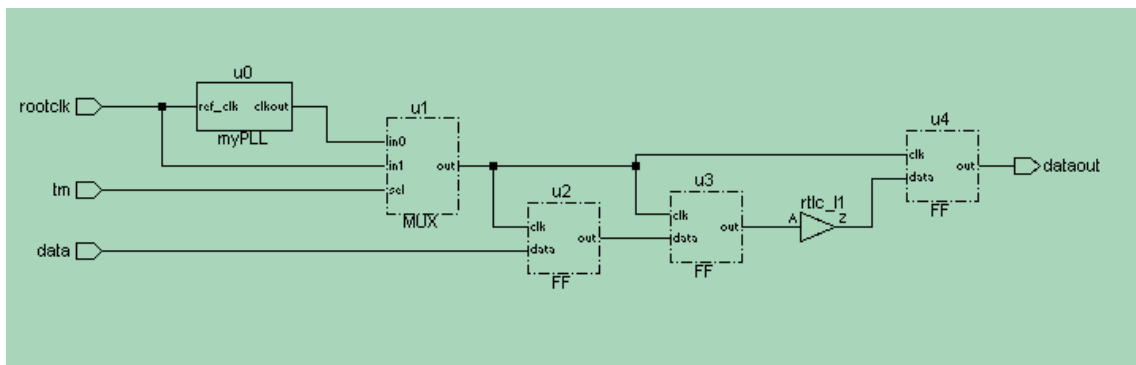


FIGURE 6. PLL Example

Clock Gating Cells

SpyGlass DSM supports clock gating cell (CGC) modules with structural descriptions as shown in [Figure 7](#), as well as black boxes defined with constraints.

CGC White Box

SpyGlass DSM has a CGC recognizer that checks for the following conditions:

- Entire gating structure should be in a module / entity boundary
- It should either have 1 or 2 enable pins.
- It should have simple / regular CGC structures like:
 - OR-Gate (for enables), Latch with inverted clock connected to enable followed by AND-Gate with latch output and clock – “Inferred Positive Edge CGC (WB_iCGC_POS)”
 - OR-Gate (for enables), Latch with same phase clock connected to enable followed by OR-Gate with inverted latch output and clock – “Inferred Negative Edge CGC (WB_iCGC_NEG)”

If these conditions are satisfied for a module, then DSM considers that module to be a CGC and all CG rules may be used for that module.

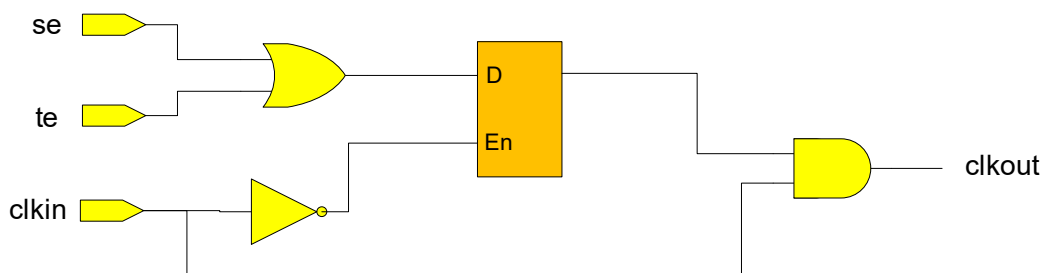


FIGURE 7. Clock gating cell example

CGC Black Box

A black box may also be declared as a CGC by using the `gating_cell` constraint. When so declared, a module may be checked by all CG rules just as if the module were defined structurally.

At-speed Clocks

At least one clock must be declared as `-atspeed`.

If the `-atspeed` option is not used with the `clock` constraint, the transition coverage calculation will not operate correctly.

`Atspeed_11` can be used to identify clock sources that are not controlled by an at-speed clock.

`Info_testclock` for `-captureATspeed` is used to display at speed clock propagation.

SGDC constraint	-testclock	-atspeed	What does it mean?
clock	No	No	Denotes clock for system use only
clock	Yes	No	Clock used for shift & stuck at capture
clock	No	Yes	Transition testing only
clock	Yes	Yes	Used for scan, static capture & transition testing

Testmode for At-speed

If the design has a by-passed PLL, as is often used for stuck-at scan design, then `test_mode` constraints are required.

As shown in [Figure 6](#), a mux (u1) is used to bypass the PLL so that scan clocks can be driven directly from the test machine. In `captureATspeed` mode, the mux must select the clock from `myPll`. To do this, the following constraints are required:

```
test_mode -name tm -value 1 -scanshift
test_mode -name tm -value 0 -captureATspeed
```

NOTE: Often, in circuits with a PLL by pass, the bypass is required for both `scanshift` and for `capture`. In such cases, if the following `test_mode` constraint is used, `tm` would be assigned the value 1 for all modes and therefore would not be correct for `captureATspeed`.

```
test_mode -name tm -value 1
```

If the following constraints are used to have `tm = 1` for `scanshift` and `capture` and 0 for `captureATspeed` then a syntax error is issued because

`test_mode` with no option means that the specified value is applied for all modes. Since the requirement for *captureATspeed* has a different value, then a syntax error is issued.

```
test_mode -name tm -value 1
test_mode -name tm -value 0 -captureATspeed
```

The correct method is to use three `test_mode` constraints:

```
test_mode -name tm -value 1 -scanshift
test_mode -name tm -value 1 -capture
test_mode -name tm -value 0 -captureATspeed
```

Rules to Identify and Verify At-speed Constraints

The *PLL_01* will identify PLLs whose clock input is not controlled by an at-speed clock and *PLL_02* will identify PLLs whose reset pin (if declared in a PLL constraint) is not controlled by a root level pin.

Info_atspeedClock: Displays at-speed test clock propagation.

Atspeed_01 will identify flip-flops whose clock input has a path that does not go through a PLL.

The rule *Atspeed_07* detects pairs of at-speed clocks that have the same gating *conditions*. If this is the case, then it may not be possible to isolate clock domains which can impact ATPG performance.

Info_testclock and *Info_testmode* are also useful to diagnose at-speed clock propagation issues.

Best Practice

This step identifies DSM issues in the design that may prevent achieving high transition fault coverage. This step also identifies issues that prevent ATPG generated tests from working as expected.

Use of the `Info_transitionCoverage_audit` Rule

The objective of the first section of this report is to fix issues so that more faults can be tested and thereby coverage is increased. The objective of the second section of this report is to ensure that ATPG generated tests

operate as expected.

Using the Coverage_Audit report to increase coverage

The rule Info_transitionCoverage_audit analyzes various design and constraint issues that can prevent achieving high transition fault coverage. The rule produces a violation message for each issue that is, in some way, blocking coverage. In addition, the rule produces a prioritized report that summarizes the results from all design and constraint issues. Each issue that has no effect on coverage (such as making all latches transparent for a design that does not contain latches) is clearly identified in the report and does not cause a violation message for that issue.

NOTE: *The first line of the report lists the coverage expected given the current state of the design and the existing constraints. Each additional line lists a possible cause for low coverage and the coverage that would be realized if that cause was removed. An example of the report is shown in [Figure 8](#).*

The following figure illustrates an example of this report:


```

Atspeed launch method 'LAUNCH_ON_CAPTURE'

Base transition fault coverage :97.8
Base transition test coverage :97.8

Expected transition coverage data after each step.

```

	Expected transition coverage		Action
	Fault coverage	Test coverage	
1. Flip-flops made scannable	97.8	97.8	No action required
2. Black-box made scanwrapped	97.8	97.8	No action required
3. 'no_scan' flip-flops made scannable	97.8	97.8	No action required
4. At speed test clocks	97.8	97.8	No action required
5. PI and PO used for launch and capture	97.8	97.8	No action required
6. Controllable flip-flop d-pins	97.8	97.8	No action required
7. Clock domain crossing	98.3	98.3	Atspeed_06 & Diagnose_04
8. 'no_atspeed' faults removed	98.3	98.3	No action required
9. False path faults	98.3	98.3	No action required
10. Atpspeed test mode	99.5	99.5	Info_testmode && Info_untestable
11. Tristate enables	99.5	99.5	No action required
12. Undetected faults due to 'force_ta'	99.5	99.5	No action required
13. Hanging terminals	99.9	99.9	TA_09
14. Undetected faults due to PG nets	100.0	100.0	Info_pwrGndSim, Info_synthRedundant & Atpspeed_

FIGURE 8. Example of the audit_report section to improve coverage

Using the Info_random_resistance rule to identify hard to test faults

The *Info_random_resistance* rule provides the capability to identify the design sections, which contain hard to test faults and may cause ATPG to abort, while generating the test vector for such faults.

You can enable the Autofix feature for a goal and for the supported rules using the `dft_autofix` and the `rme_active` parameters.

The following snippet from the project file describes method to enable the autofix feature for the *Info_random_resistance* rule:

```

current_goal dft/dft_dsm_best_practice
set_goal_option addrules Info_random_resistance

```

For more information on running the Selective Autofix feature, refer to the *Running the Selective Autofix* section in the *SpyGlass DFT DSM Rules Reference Guide*.

D-pin Controllability

The DFT_DSM rule *Atspeed_09*, highlights the root cause for bad controllability at the data pin of a scannable flip-flop.

Possible causes for lack of control are:

- D-input to launch FF = X caused by black boxes
- D-input to launch FF = X caused by non-scan flip-flops
- D-input to launch FF = X caused by unconnected net
- D-input to launch FF has forced value
- D-input to launch FF has multiple drivers

The *Atspeed_09* rule identifies the cause and highlights the schematic accordingly.

Flip-flop set/reset pin controllability

The *Atspeed_20* rule, highlights the flip-flop and the path from the asynchronous pin that is not fully controllable.

For possible causes for lack of control, see the description for D-pin controllability.

Flip-flop set/reset pins fed by re-convergence

The *Atspeed_25* rule, highlights the flip-flop and the path from the asynchronous pin to the re-convergence logic as illustrated below.

Methods to fix:

- Modify the re-convergence logic so that only a single path is feeding the flip-flop
- Modify the re-convergence logic so that only one single path is feeding the flip-flop in capture*Atspeed* mode:

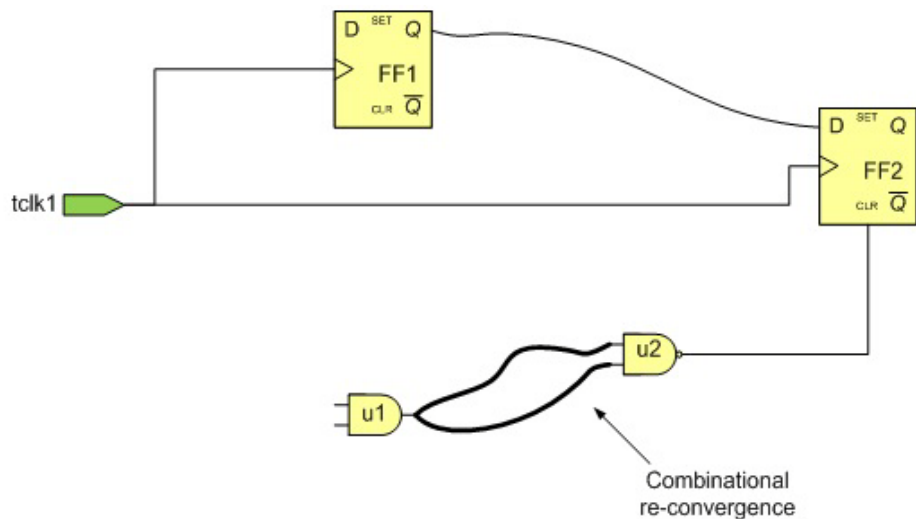


FIGURE 9. Combinational Re-convergence

Latch Transparency

Since ATPG tools operate most efficiently for combinational circuits, it is important that latches (with the exception of re-timing latches) be forced to their enabled condition so that they may be treated as if they were a buffer during *captureATspeed* conditions. Rule *Atspeed_19* can be used to identify all latches that are not transparent when *captureATspeed* conditions are applied or when testclocks are inactive.

Use PIs and POs for Launch and Capture

In cases where the test machines can create the necessary transitions on primary inputs, see [Figure 10](#). If PIs are used as launch points then faults in the combinational logic between the primary inputs and the first level of registration flip-flops can be transition tested. SpyGlass DFT DSM has a parameter `dsmUsePIForAtSpeed` that determines whether or not PIs can be used as launch points. The default value is on.

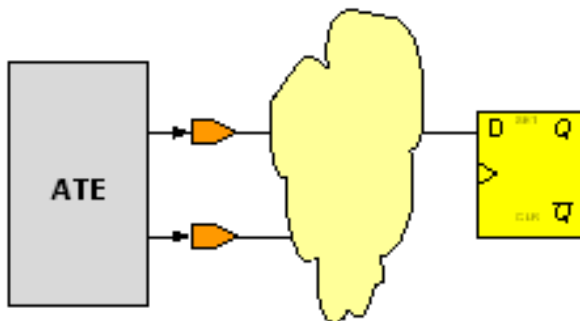


FIGURE 10. PIs used as launch points

The `dsmUsePOForAtSpeed` parameter determines whether or not POs can be used as capture points. See [Figure 11](#). PO used as capture point. The default value is on.

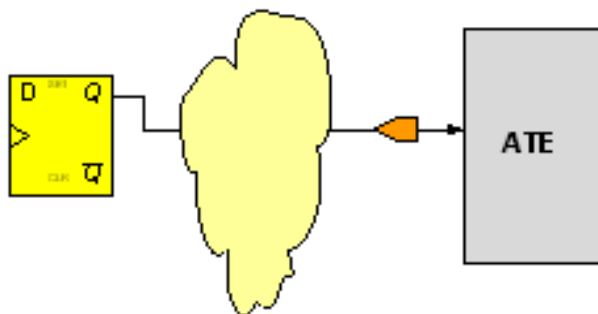


FIGURE 11. PO used as capture point

Asynchronous Domains that Become Synchronous

DFT_DSM rule *Atspeed_03*: Asynchronous logic in the functional mode should not interact synchronously in the atspeed test mode. See [Figure 12](#).

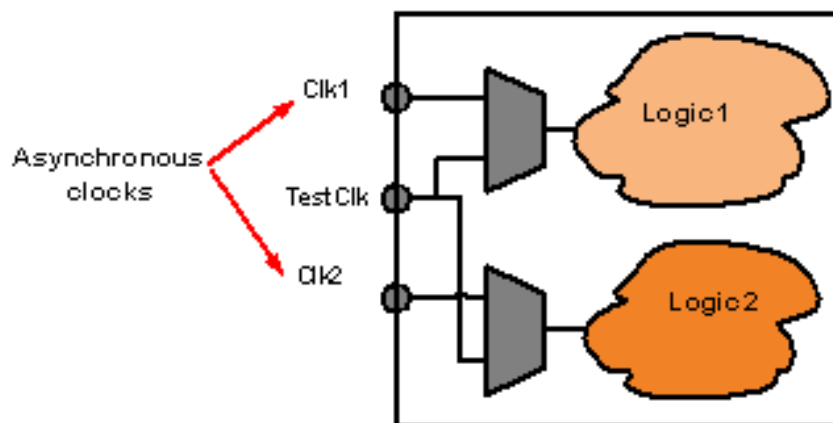


FIGURE 12. Asynchronous system domains become synchronous test domains

Synchronous Domains that Become Asynchronous

DFT_DSM rule: *Atspeed_04*: Synchronously interacting logic in the functional mode should not be clocked by asynchronous clocks in the test mode. See [Figure 13](#).

These are the faults that cannot be transition tested because they are driven in one clock domain but can only be observed in another domain.

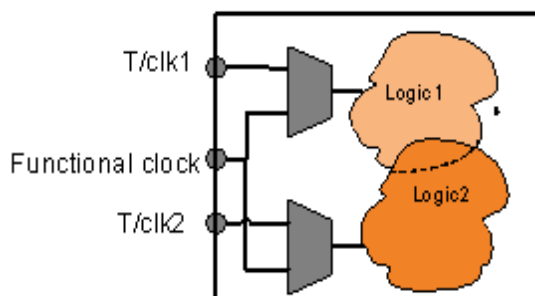


FIGURE 13. Synchronous system domains become asynchronous test domains

Solve Path Issues

Atspeed_05: Blocked false paths or multi-cycle paths in the functional mode

Atspeed_06: Paths crossing the asynchronous clock domains that are not blocked

Diagnose_02: Number of faults that are blocked under capture at-speed mode simulation conditions in the design

Diagnose_03: Number of faults that are blocked by false paths or multi-cycle paths in the design

Diagnose_04: Number of faults that cannot be tested in capture at-speed mode due to crossing of clock domains

Estimate Transition Fault Coverage

Eva

luates and reports transition coverage for design. As explained in the [Transition Test Design](#) section.

Testing requires a clock pulse to change the state of the launch flip-flop. In order for the launch flip-flop to produce both rising and falling transitions, the D flip-flop must be controllable to both a 1 and a 0.

Step-by-Step Solution

This section describes the steps for using SpyGlass DSM in some detail.

Goal: `dft_scan_ready`

Run the `scan_ready` goal to define various test constraints. When identifying test clocks through the GUI, you can also indicate which of the clocks are `atspeed` clocks for transition testing.

Use this goal to maximize scannability of a design. This step is critical for high transition coverage.

PLL_01 violations

PLL_01 violations indicate three possibilities: missing `atspeed` clock constraints, missing `test_mode` constraints to enable the path from an `atspeed` clock to the PLL clock input or improper logic connected to the PLL input clock

- In each of these cases, display a violation in the incremental schematic viewer (IS), by clicking on the violation message to select it, and then clicking on the IS icon as shown in the following figure:

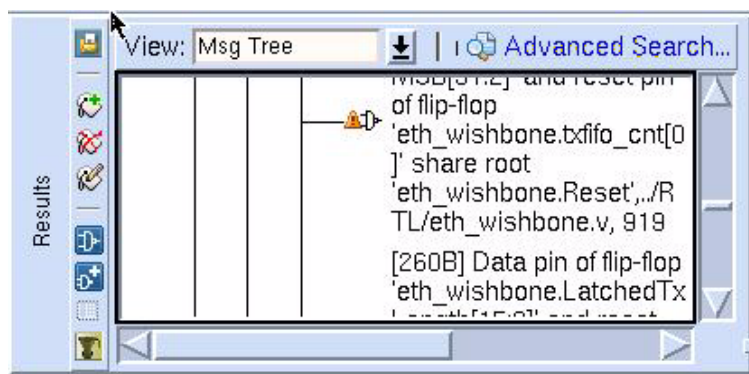


FIGURE 14. PLL_01 example

- Traverse the PLL input clock cone backward toward root level inputs. One of these should be declared as an atspeed testclock
- If the path to an atspeed clock exists but is not enabled, then add `test_mode -atspeed`
- If no path is found, then consider a logic change that allows the PLL clock input to be controlled by a root level atspeed testclock

PLL_02 violations

PLL_02 violations indicate three possibilities: missing reset constraints, missing `test_mode` constraints to enable the path from a reset to the PLL reset input or improper logic connected to the PLL reset input

- In each of these cases, display a violation in the IS.
- Traverse the PLL reset input cone backward toward root level inputs. One of these should be declared as a reset
- If the path to a reset exists but is not enabled, then add appropriate `test_mode -atspeed` to enable the path.
- If no path is found, then consider a logic change and associated `test_mode -atspeed` constraints to allow the PLL reset input to be controlled by a root reset pin.

Atspeed_01 violations

Atspeed_01 violations indicate flip-flops with clocks not controlled by a PLL output clock in atspeed mode.

- The display for an Atspeed_01 violation in the IS shows a flip-flop and a path back to a clock
- The logic for this path must be modified to include a PLL or, a PLL constraint should be added to some device on the displayed clock path.
- `Info_atspeedClock` may also help to debug these situations.

Atspeed_07 violations

Atspeed_07 violations identify atspeed clocks with gating signals in common.

- Use the IS to display the testclocks and a signal that is used to enable both testclocks
- Right clicking on the highlighted enable signal gives options to display the signal's fanout cone
- The violation may be corrected by changing the clockGating constraints or by changing the logic to prevent the same enable from controlling both testclocks

NOTE: *This rule will only operate if clockGating constraints are used to specify clock gating cell enabling signals.*

Atspeed_11 violations

Ensure that all clock sources are controlled by an atspeed clock.

- Atpspeed_11 violations identify clock sources that are not driven by an atspeed clock
- View a violation in the IS and display the fanin cone to the highlighted clock source.
- Use the Debug Data window to locate a clock with a path to this source.
- If such a clock exists then declare this as an atspeed clock
- If no clock is found then either a clock constraint must be added or the design must be modified

CG_01_shift violations

CG_01 violations identify clock gating cells with values on the enable pins that prevent clock propagation in shift mode

NOTE: *SpyGlass allows either 1 or 2 enable pins. If 2 enables are used, then SpyGlass assumes that the 2 enables are OR'ed inside the CGC module. CG_01_shift will flag a violation if either enable pin cannot be at logical 0 during scan shift mode.*

- The display of a *CG_01_shift* violation in the Incremental Schematic will show the clock gating cell and the test enable pin(s), with the non-enabling values, will be highlighted
- Right clicking a highlighted pin brings up a sub menu with various choices for displaying the pin's fanin cone
- These choices allow for displaying the entire cone or just a path through the cone
- In either case, any signal with a value forced in shift mode will be displayed
- Either a constraint must be changed or the fanin cone design must be changed

CG_01_capture violations

CG_01_capture violations identify clock gating cells with values on the enable pins that prevent clock propagation in capture mode

NOTE: *SpyGlass allows either 1 or 2 enable pins. If 2 enables are used, then SpyGlass assumes that the 2 enables are OR'ed inside the CGC module. CG_01_capture will flag a violation if either enable pin cannot be at logical 0 during capture mode.*

- The display of such violations in the IS will show the clock gating cell and the test enable pin(s), with the non-enabling values, will be highlighted
- Right clicking a highlighted pin brings up a sub menu with various choices for displaying the pin's fanin cone
- These choices allow for displaying the entire cone or just a path through the cone
- In either case, any signal with a
- value forced in capture mode will be displayed
- Either a constraint must be changed or the fanin cone design must be changed

CG_01_atspeed

CG_01_atspeed violations identify clock gating cells with values on the enable pins that prevent clock propagation in atspeed mode

NOTE: *SpyGlass allows clock gating cells with either 1 or 2 enable pins. If 2 enables are used, then SpyGlass assumes that the 2 enables are OR'ed inside the CGC module. CG_01_atspeed will flag a violation if either enable pin cannot be at logical 0 during atspeed mode.*

- The display of such violations in the IS will show the clock gating cell and the test enable pin(s), with the non-enabling values, will be highlighted
- Right clicking a highlighted pin brings up a sub menu with various choices for displaying the pin's fanin cone
- These choices allow for displaying the entire cone or just a path through the cone
- In either case, any signal with a value forced in atspeed mode will be displayed
- Either a constraint must be changed or the fanin cone design must be changed

CG_02_capture violations

CG_02_capture violations identify clock gating cells with enable pins that cannot be controlled to the disabled state in capture mode

NOTE: *SpyGlass allows clock gating cells with either 1 or 2 enable pins. If 2 enables are used, then SpyGlass assumes that the 2 enables are OR'ed inside the CGC module. CG_02_capture will flag a violation if either enable pin cannot be at logical 0.*

- The display of such violations in the IS will show the clock gating cell and highlight the test enable pin(s) that cannot be disabled
- Right clicking a highlighted pin brings up a sub menu with various choices for displaying the pin's fanin cone
- These choices allow for displaying the entire cone or just a path through the cone
- In either case, any signal with a value forced in capture mode will be displayed
- Either a constraint must be changed or the fanin cone design must be changed

CG_02_atspeed violations

CG_02_atspeed violations identify clock gating cells with enable pins that cannot be controlled to the disabled state in atspeed mode

NOTE: *SpyGlass allows clock gating cells with either 1 or 2 enable pins. If 2 enables are used, then SpyGlass assumes that the 2 enables are OR'ed inside the CGC module. CG_02_atseed will flag a violation if either enable pin cannot be controlled to logical 0.*

- The display of such violations in the IS will show the clock gating cell and highlight the test enable pin(s) that cannot be disabled
- Right clicking a highlighted pin brings up a sub menu with various choices for displaying the pin's fanin cone
- These choices allow for displaying the entire cone or just a path through the cone
- In either case, any signal with a value forced in capture mode will be displayed
- Either a constraint must be changed or the fanin cone design must be changed

CG_03_atspeed violations

CG_03_atspeed violations identify clock gating cell se pins that are not observable

NOTE: *Unobservability for a CGC se pin is based on the controllability of the other enable pin (if any) and the observability of the CGC output pin.*

- Select a CG_03_atspeed violation and display it in the Incremental Schematic
- The clock gating cell outline and the test enable pin that cannot be observed is highlighted
- Other enables: Right-click any other enable pin. Hover cursor on **Show input cone** and click on **To primary inputs** to see the fanin logic driving this enable

CG_generateReport

CG_generateReport causes a report called `dft_dsm_clock_gating_cell.rpt` to be generated. This report lists all the clock gating cells found in the design along with a variety of data about each cell including the number of flip-flops controlled by each and all CG rules violations reported for each cell.

- Use this report to ensure that all clock gating cells in the design have been found

Info_atSpeedClock

Info_atSpeedClock displays the propagation paths for all system clocks and atspeed clocks. This display is often useful when diagnosing clock related rules.

- Use this display to trace specific clock trees
- Use the DFT Debug Data window to see values on clock enable pins

Goal: dft_dsm_best_practice

- Run the `dft_dsm_best_practice` goal
- Review the `transition_coverage_audit` report

Info_transitionCoverage_audit

This informational rule computes what the fault coverage would be if the major causes of low transition fault coverage had been eliminated. This rule takes each fault category in the following list and computes the new coverage assuming that the faults in that category were detectable.

This information should be used as a prioritized guide for determining the problems that should be addressed to improve coverage.

- *Audit category: Make all flip-flops scannable*
- *Audit category: Add scan wrappers to black boxes*

- *Audit category: Consider faults blocked by "no-scan" faults as testable*
- *Audit category: Improve launch and capture by adding atspeed clocks*
- *Audit category: Use all PIs and Pos for launch and capture*
- *Audit category: Make all scan flip-flop d-pins controllable*
- *Audit category: Block paths that cross clock domains*
- *Audit category: Treat faults marked "no-atspeed" as testable*
- *Audit category: Treat faults in false and multi-cycle paths as testable*
- *Audit category: Unblock faults blocked by Atspeed testmode constraints*
- *Audit category: Tristate enables*
- *Audit category: Remove effects of "force_ta" constraints*
- *Audit category: Treat hanging terminals as fully controllable*
- *Audit category: Treat faults blocked by power and ground nets*

The report indicates the current coverage for the design and the steps that you can follow to improve the coverage. The following sections, listed in report order, describe the diagnostic procedure for each fault category in the coverage improvement section of the report.

Audit category: Make all flip-flops scannable

- Scannability is critical to achieving high transition coverage
- The step by step instructions for making flip-flops scannable is covered in the *Make flip-flops scannable* section of the *SpyGlass DFT Methodology* guide.

Audit category: Add scan wrappers to black boxes

- The step by step instructions for adding scan wrappers to black boxes is covered in the *Scan-wrap black boxes* section of the *SpyGlass DFT Methodology* guide.

Audit category: Consider faults blocked by "no-scan" faults as testable

- The step by step instructions for handling faults rendered untestable due to "no-scan" constraints is covered in the *"no-scan" flip-flops made scannable* section of the *SpyGlass DFT Methodology* guide.

Audit category: Improve launch and capture by adding atspeed clocks

Atspeed_11 violations identify clock sources that are not controlled by an atspeed testclock during atspeed capture

- Select an Atspeed_11 violation and display in IS
- The atspeed_11 violation lists an atspeed clock source that is not atspeed clock controlled in atspeed capture mode. The message also lists that number of flip-flops controlled by that source
- The IS will display the atspeed clock source and the path from that source to one flip-flop in the domain of that source
- If the pins on the clock source are not displayed, double click anywhere on the module
- Right click on the clock input pin on the clock source
- Select "Show input cone" or "Trace to Driver"
- If a clock pin is displayed, right click on that pin and use "Set SGDC constraints" to declare that clock as an atspeed testclock
- If a clock pin is not found, consider a design change to the fanin cone to the clock source clock input pin

Audit category: Use all PIs and Pos for launch and capture

- The parameters dsmUsePIForAtspeed and dsmUsePOForAtspeed are both defaulted "on" so, under default conditions, this step will never show a coverage increase
- If there is a coverage increase then one or both of these parameters have been switched "off"
- In that case, consider resetting the parameters to the "on" state or possibly redesign the logic so these pins can be controlled by test logic

Audit category: Make all scan flip-flop d-pins controllable

- Atspeed_09 violations identify flip-flop d-pins that are not fully controllable and therefore cannot be used to launch rising edge tests, falling edge tests or both.
- Select an Atspeed_09 violation and display in IS
- If the cause of the uncontrolled d-pin is a combinational loop, then verify in the Schematic Viewer whether a testmode is missing or has been improperly declared. Otherwise, the design may have to be modified to allow use of a testmode to break the loop.

- If the cause of the uncontrolled d-pin is a multiple driven net/floating net, then verify in the Schematic Viewer whether a testmode is missing or has been improperly declared.
- If the cause of the uncontrolled d-pin is a black box, then create a scanwrapper for the module.
- If the cause of the uncontrolled d-pin is a non-lockup/Non-transparent latch, then verify in the Schematic Viewer whether a testmode is missing or has been improperly declared.
- If the cause of the uncontrolled d-pin is a non-scan flip-flop then reconsider the reasons for this to not be scannable. A quick fix is to create a “scan” constraint for this flip-flop to determine the coverage impact.
- If the cause of the uncontrolled d-pin is a constant value net has been reached, then verify in the Schematic Viewer whether a testmode is missing or has been improperly declared.

Audit category: Block paths that cross clock domains

- Double click an *Atspeed_06* violation to view a spreadsheet for the flip-flop listed in the violation message.
- The spreadsheet has a row for each flip-flop with an unblocked path from the source flip-flop.
- Click on a source domain and display in the IS which will highlight a path from this source flip-flop that crosses a clock domain that is not blocked
- Hold down the `ctrl` key and click on *Diagnose_04* to display faults on the selected path that cannot be tested because of the domain crossing
- Click on various path inputs to determine places where the blocking logic can be added

Audit category: Treat faults marked “no-atspeed” as testable

- Select an *Info_noAtspeed* and display in the IS
- All faults on this entity are excluded from testing and therefore contribute to a coverage
- Consider modifying or deleting the `no_atspeed` constraint on this entity if testing is required

Audit category: Treat faults in false and multi-cycle paths as testable

- Double click an *Atspeed_05* violation to view a spreadsheet for the paths listed in the violation message.
- The spreadsheet has a row for each unblocked multi cycle or false path
- Click on a path and display in the IS which will highlight an unblocked path
- Hold down the `ctrl` key and click on *Diagnose_03* to display faults on the multi-cycle path that cannot be tested
- Click on various path inputs to determine places where the blocking logic can be added

Audit category: Unblock faults blocked by Atspeed testmode constraints

- Select an *Atspeed_19* violation in the IS. Display the enable fan-in cone back to primary inputs to see if or how this logic should be enabled in captureATspeed mode
- Display an *Diagnose_02* violation with the IS.
- The blocking gate and the blocked pin on this gate will be highlighted
- Use the DFT Debug Data window to see the value on this pin
- Display the fanin cone to the blocking pin and use the DFT Debug Data window to see values on fanin pins as necessary to determine if a constraint should be changed or if the design should be modified

Audit category: Tristate enables

The step by step instructions for making tristate enable pins observable is covered in the *Tristate nets made observable* section of the *SpyGlass DFT Methodology* guide.

Audit category: Remove effects of “force_ta” constraints

The step by step instructions for removing the effects of “force_ta” constraints is covered in the “force_ta” nets and “test_point” constraint pins made testable section of the *SpyGlass DFT Methodology* guide.

Audit category: Treat hanging terminals as fully controllable

The step by step instructions for treating hanging terminals as fully controllable is covered in the *Hanging nets made controllable* section of the *SpyGlass DFT Methodology* guide.

Audit category: Treat faults blocked by power and ground nets

The step by step instructions for treating faults blocked by power and ground nets as controllable is covered in the *Testmode/Tied pins made controllable* section of the *SpyGlass DFT Methodology* guide.

Atspeed_03 Violations

- Select an Atspeed_03 violation and display in the IS
- The IS highlights the path that crosses a clock domain boundary
- The violation can be fixed by either blocking the domain crossing path or by modifying the testclock distribution logic
- Path blocking: Hover the cursor on a path enable to check the controllability and simulation values
- Path blocking: If necessary, right-click on a path enable input to display the input fan-in cone logic
- Path blocking: Consider modifying the re-convergence logic so that one of the re-convergence paths is disabled during atspeed capture
- Test clock modification: The Atspeed_03 violation identifies the testclock that combines the asynchronous functional clock domains.
- Test clock modification: find this testclock as a violation for the Info_atSpeedClock rule
- Test clock modification: display this violation in the IS will show how this testclock feeds into the two functional clocks
- Test clock modification: consider replacing the testclock with two testclocks so that the two original asynchronous functional domains remain asynchronous in atspeed test mode

Atspeed_04 Violations

- Select an Atspeed_04 violation and display in the IS
- The IS will highlight a path that crosses a clock domain boundary and will identify the functional clock name
- Use one of the following methods to fix the Atspeed_04 violations:

- ☐ Modify the testclock distribution logic so that only one testclock is used for this functional clock
- ☐ Use the -domain to combine the testclocks
- ☐ Block all domain crossing paths
- Test clock modification: The Atspeed_04 violation identifies the testclock that combines the asynchronous functional clock domains
- Test clock modification: Find this testclock as a violation for the Info_atSpeedClock rule
- Test clock modification: display this violation in the IS will show how this testclock feeds into the two functional clocks
- Test clock modification: consider replacing the testclock with two testclocks so that the two original asynchronous functional domains remain asynchronous in atspeed test mode
- Path blocking: NOTE for this approach to work, all paths that cross test clock domains must be blocked. Since there may be many such paths, this may not be practical
- Path blocking: Hover the cursor on a path enable to check the controllability and simulation values
- Path blocking: If necessary, right click on a path enable input to display the input fanin cone logic
- Path blocking: Consider modifying the re-convergence logic so that one of the re-convergence paths is disabled during atspeed capture

Atspeed_05 Violations

- Double click an Atspeed_05 violation
- Double click on a false path in the spreadsheet and display in the IS
- Hover the cursor over any inputs to the path to determine their current values
- Right-click on an input and select a fan-in cone to see the logic driving an input if necessary
- Add a testmode constraint to force a path enable to a blocking value or consider modifying the enable logic

Atspeed_06 Violations

- Select an *Atspeed_06* violation to see spreadsheet of paths that cross clock domains. Each row corresponds to a path.
- Select a row and then select IS to “see” the path.
- If the domains should be considered as synchronous, then consider use of clock_group constraint to merge atspeed testclocks
- If the domains are asynchronous, then consider blocking the path with a design change.
- Use Diag_04 to highlight faults that cannot be detected because of domain crossing paths.

Atspeed_19 Violations

- Select an *Atspeed_19* violation and display in the IS.
- The IS will show a latch with the clock pin highlighted
- The violation can be fixed by inverting the OFF state of the clock constraint or changing the inversion parity of the path from the clock to the latch pin

Atspeed_20 Violations

- Select an *Atspeed_20* violation and display in the IS.
- The IS will show a flip-flop with the async pin and fanin path highlighted
- The IS will also display the controllability of the async pin
- If the controllability is ny- then controllability to 0 is the problem
- If the controllability is yn- then controllability to 1 is the problem
- If the controllability is nn- then the pin cannot be made either 0 or 1
- Right click on inputs to the displayed path and then click on “Show Debug Data” and “DFT”
- The pop-up window will show the controllability of that pin
- Extend the fanin display by right clicking a path input and selecting “Show fanin cone”

- Either the design must be changed by possibly adding a test point or, if a non-scanned flip-flop has been reached then consider making it scannable
- If the path controllability is caused by a constraint, then consider modifying or removing the constraint

Atspeed_25 Violations

- Double click an Atspeed_25 violation and display in the IS
- The flip-flop and the reconverging paths to the reset/set pin will be displayed
- Hover the cursor over a path enable to see the values on this pin
- Right-click a reconverging path enable pin and select "show input cone" to display the logic driving this enable
- Consider adding a testmode constraint to disable a reconverging path
- Also consider modifying a reconverging path so that it can be blocked during atspeed capture

Diagnose_03 Violations

Diagnose_03 may be used to view faults that are involved in false paths or multi-cycle paths.

NOTE: *If the design has many false paths or multi-cycle paths, then displaying the results of this rule may involve a large number of faults and therefore may not be practical. Consider use of AtSpeed_05.*

Diagnose_04 Violations

Diagnose_04 may be used to view pins with faults that cannot be tested because they involve paths that cross clock.

NOTE: *If the design has many paths crossing clock domains, then displaying the results of this rule may involve a large number of faults and therefore may not be practical. Consider use of AtSpeed_06.*

Info_transitionCoverage

This informational rule calculates the atspeed transition fault coverage. The results may be used to determine modules where testability changes may be useful. Refer to the SpyGlass-DFTDSM Reference guide for more details.

- *Info_transitionCoverage* may be used to estimate the transition fault coverage if ATPG were run based on the current circuit and SGDC constraints.
- If the coverage estimate reported falls below expectation, run *Info_transitionCoverage_audit* to find the major causes of low atspeed test (transition delay) coverage.

Using the Coverage_Audit report to ensure test operation

The second section of the Audit-Coverage report covers rules that detect conditions which, if not fixed, can prevent ATPG tests from operating as expected. As a result, actual coverage may be less than the result predicted by ATPG tools or tests may fail even when a chip is operating correctly.

Rules used in the second section of the audit report:

- Atspeed_03
- Atspeed_04
- Atspeed_05
- Atspeed_14
- Atspeed_22
- Atspeed_25
- Atspeed_30

This section of the report is sorted by the number of flip-flops affected in each rule. Flip-flop count is used instead of computing possible change in coverage because of runtime consideration.

Atspeed_03 violations

See the [Best Practice](#) section for details of the *Atspeed_03* rule.

Atspeed_04 violations

See the [Best Practice](#) section for details of the *Atspeed_04* rule.

Atspeed_05 violations

See the [Best Practice](#) section for details of the *Atspeed_05* rule.

Atspeed_14 violations

- Double click an Atpspeed_14 violation
- Double click on a clock-as-data row and display in the IS
- Hover the cursor over any inputs to the path feeding flip-flop data pin to determine their current values
- Right click on an input and select a fanin cone to see the logic driving an input if necessary
- Add a testmode constraint to force a path enable to a blocking value or consider modifying the enable logic

Atspeed_22 violations

- Double click an Atpspeed_22 violation
- Double click on a clock-as-data row and display in the IS
- Hover the cursor over any inputs to the path feeding flip-flop data pin to determine their current values
- Right click on an input and select a fanin cone to see the logic driving an input if necessary
- Add a testmode constraint to force a path enable to a blocking value or consider modifying the enable logic

Atspeed_25 violations

See the [Best Practice](#) section for details of the *Atspeed_25* rule.

Atspeed_30 violations

- Double click an Atpspeed_30 violation
- The flip-flop and the reconverging paths to the clock pin will be displayed
- Hover the cursor over a path enable to see the values on this pin
- Right click a reconverging path enable pin and select "show input cone" to display the logic driving this enable
- Consider adding a testmode constraint to disable a reconverging path
- Also consider modifying a reconverging path so that it can be blocked during atspeed capture